HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

SOICT

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.

LLMs for Software Engineering

2. Predictive Analytics and Data-driven Development

ONE LOVE. ONE FUTURE.

FYI

- Lecturer: Dr. Bui Thi Mai Anh
- **Research domains:**
  - Software Engineering
  - Applied AI in Software Engineering
  - Search-based Software Engineering**: evolutionary computing, reinforcement learning, deep learning**
  - **NLP4SE**
- Lab: **Intelligent Software Engineering** – BK AI Center

https://iselab-dorlab.github.io/

Predictive Analytics and Data-Driven in SE

- Application of data-driven techniques to forecast future events or behaviors in the software development lifecycle

- Typically powered by statistics, machine learning, and AI

- The goals:
  - anticipate risks
  - improve planning
  - enhance decision-making throughout development and maintenance

Topics of interest

- **Probability of software defects**
  - Predicting whether a software module, class, or feature is likely to contain defects before release
- **Delivery delays**
  - Forecasting whether a software project, sprint, or milestone will be completed on schedule
- **Cost overruns**
  - Predicting when actual project cost will exceed the planned budget
- **System reliability issues**
  - Predicting failures, downtime, or degraded performance in deployed systems
- **Developer productivity**
  - Forecasting individual or team productivity to identify bottlenecks, burnout risk, or optimal allocation

What is the data in these topics?

- **Historical data** (e.g., past project metrics, defect logs, productivity patterns) to build models that **predict future outcomes**
- **Some critical data categories:**
  - Project and Process data
  - Product (Code & Design) Data
  - Quality & Defect Data
  - Operational & Usage Data
  - Team & Human Factors Data
  - Repository & Tool Data
  - External Benchmark Data

Project & Process Data

These capture the **management side** of development

- **Effort Logs**: Person-hours spent on tasks, sprints, or features.
- **Schedule Data**: Planned vs. actual start/finish dates, milestones, delays.
- **Resource Allocation**: Number of developers assigned, skill levels, overtime patterns.
- **Cost Records**: Budgeted vs. actual expenditures across phases.
- **Change Requests**: Frequency, size, and impact of scope changes.
- **Past Estimations**: Historical estimates and the actual outcomes (used to calibrate effort/cost prediction)

Product (Code & Design) Data

These describe the software artifacts themselves:

- **Code Complexity Metrics**: Cyclomatic complexity, depth of inheritance, lines of code (LOC).
- **Size Metrics**: Function points, number of modules/classes/methods.
- **Code Churn**: Frequency and size of changes (commits, deletions, additions).
- **Static Analysis Reports**: Detected vulnerabilities, coding standard violations.
- **Dependency Graphs**: Coupling, cohesion, and modularization indicators.

Quality & Defect Data

Essential for **defect prediction and reliability modeling**:

- **Bug Reports**: Number, severity, type (functional, security, performance).
- **Defect Density**: Defects per KLOC (thousand lines of code).
- **Time to Fix**: Duration from bug discovery to resolution.
- **Failure Logs**: Crash reports, runtime errors, test failures.
- **Historical Test Results**: Pass/fail patterns across regression test suites.
- **Code Review Findings**: Issues raised and resolved during peer review.

Operational & Usage Data

These capture **runtime behavior**:

- **System Logs**: CPU, memory usage, I/O bottlenecks, error logs.
- **Incident Reports**: Outages, downtime duration, MTTR (mean time to repair).
- **User Behavior**: Usage patterns, feature adoption rates.
- **Telemetry Data**: Performance monitoring from deployed software.

Team & Human Factors Data

Important but often overlooked:

- **Developer Activity**: Commit frequency, pull request cycle times.
- **Team Dynamics**: Collaboration metrics from tools like Jira, GitHub, Slack.
- **Turnover & Experience**: Developer skill profiles, historical team changes.
- **Workload Distribution**: Balance of assigned tasks among team members.

Repository & Tool Data

Sources that act as **data reservoirs**:

- **Version Control Systems (Git, SVN)**: Commit history, branches, merges.
- **Issue Trackers (Jira, Bugzilla)**: Ticket histories, priorities, SLA compliance.
- **CI/CD Pipelines**: Build failures, deployment frequency, test coverage.
- **Documentation Systems**: Requirements evolution, traceability logs.

External Benchmark Data

Sometimes combined with internal history

- Industry Repositories: PROMISE dataset, NASA MDP for defect prediction.
- Benchmarks: Effort estimation datasets (COCOMO, ISBSG).

"Historical data in software engineering spans ***project management, product code, quality metrics, runtime operations, human/team factors, and tool repositories****."*

Application: Defect Prediction

Objectives:

- Identifying modules or components most likely to contain bugs.
- Uses data such as code complexity, commit history, or developer activity.
- Helps prioritize testing and code reviews.

Application: Defect Prediction (2)

Data Sources (Historical)

- **Code metrics**: Lines of code (LOC), cyclomatic complexity, code churn.
- **Defect logs**: Past bugs per module, severity levels, density.
- **Review/Test history**: Number of issues found in past code reviews and unit tests.
- **Repository activity**: Commit frequency, size of changes, developer ownership.

Application: Defect Prediction (3)

Techniques

- Logistic regression, Random Forest, Neural Networks.
- Classification models flagging “defect-prone” vs. “non-defect-prone” modules.

Application: Defect Prediction (4)

Why It Matters?

- Helps QA teams **prioritize testing** on high-risk modules.
- Reduces post-release bugs, improving customer trust.
- Saves cost by catching issues early (defect fix cost grows exponentially over lifecycle).

Application: Effort & Cost Estimation

Objectives:

- Forecasts development effort (time, resources, cost) based on past projects.
- Improves planning accuracy and prevents budget overruns.

Application: Effort & Cost Estimation (2)

Data Sources (Historical):

- **Schedule records**: Planned vs. actual delivery times of past projects.
- **Effort logs**: Actual hours worked compared to estimates.
- **Change requests**: Frequency and scope changes during development.
- **Team factors**: Developer availability, turnover, productivity levels.
- **Tool data**: Jira sprint burndown charts, unfinished tasks.

Application: Effort & Cost Estimation (3)

Techniques

- Time-series forecasting (ARIMA, Prophet).
- Survival analysis to estimate probability of delay.
- ML models correlating backlog, changes, and velocity with missed deadlines.

Application: Effort & Cost Estimation (4)

Why It Matters?

- Enables **realistic project planning**.
- Prevents costly contractual penalties.
- Improves **customer satisfaction** by reducing broken delivery promises.

Application: Cost Estimation

Objectives

- Predicting when actual project cost will exceed the planned budget.

Data Sources (Historical)

- **Budget records**: Planned vs. actual costs in past projects.
- **Effort logs**: Person-hours tracked vs. estimated.
- **Resource allocation**: Overuse of senior developers, overtime patterns.
- **Change management**: Scope creep data, requirement volatility.
- **Procurement & Licensing**: Unexpected third-party costs.

Application: Cost Estimation (2)

Techniques

- Regression models (cost drivers: size, complexity, team size).
- Bayesian Networks for probabilistic cost prediction.
- Monte Carlo simulations for uncertainty analysis.

Why It Matters

- Enables **early budget correction**.
- Supports transparent stakeholder communication.
- Avoids “death march projects” where costs spiral out of control.

Application: System Reliability Issues

Objective:

- Predicting failures, downtime, or degraded performance in deployed systems.

Data Sources (Historical)

- **Failure logs**: MTBF (Mean Time Between Failures), MTTR (Mean Time To Repair).
- **Incident reports**: Outages, severity, duration.
- **Runtime telemetry**: CPU/memory spikes, error rates.
- **Defect density**: Bug-related crash history.
- **Test coverage & pass rates**: Correlation between weakly tested modules and runtime failures.

Application: System Reliability Issues (2)

Techniques

- Reliability growth models (Jelinski-Moranda, Musa-Okumoto).
- Survival analysis for time-to-failure prediction.
- Deep learning on log/telemetry data (LSTM networks for anomaly detection).

Why It Matters

- In critical domains (banking, healthcare, aviation), **downtime is unacceptable**.
- Predicting failures allows **proactive maintenance** and SLA compliance.
- Improves **end-user trust** and reduces operational costs.

Application: Developer Productivity

Objective

- Forecasting individual or team productivity to identify bottlenecks, burnout risk, or optimal allocation.

Data Sources (Historical)

- **Commit activity**: Frequency, size, code review turnaround.
- **Task completion**: Jira/Trello tickets closed vs. assigned.
- **Workload distribution**: Balance across team members.
- **Quality indicators**: Defects introduced per developer, fix rate.
- **Collaboration metrics**: PR (Pull Request) comments, pair programming logs.

Application: Developer Productivity (2)

Techniques

- Regression models linking tasks completed with influencing factors.
- Clustering developers by contribution patterns.
- Predictive dashboards combining activity + defect rates.

Why It Matters

- Helps project managers allocate work fairly.
- Detects early signs of **burnout or disengagement**.
- Identifies productivity boosters (e.g., tools, training, pair programming).
- Balances **speed vs. quality** in delivery.

Solutions?

- **Statistical Methods**: Regression analysis, time-series forecasting.
- **Machine Learning**: Decision trees, random forests, SVM, deep learning.
- **Process Mining**: Analyzing workflows to find inefficiencies.
- **Text Mining & NLP**: Processing bug reports, documentation, and commit messages.

Benefits

- Early detection of risks (defects, delays, failures).
- More accurate planning and cost estimation.
- Optimized testing strategies.
- Better allocation of human and technical resources.
- Data-driven decision-making.

Challenges

- **Data quality**: Incomplete, inconsistent, or biased historical data can distort predictions.
- **Model interpretability**: Some ML models act as “black boxes,” making it hard to justify decisions.
- **Dynamic environments**: Agile/DevOps projects evolve quickly, making past data less reliable.
- **Adoption barriers**: Requires cultural shift—teams must trust and act upon model outputs.

Trends & Future

- Integration with **DevOps pipelines** for real-time predictive insights.
- Use of **AI copilots** to recommend fixes and forecast risks on the fly.
- Growing focus on **explainable AI (XAI)** for transparency in predictions.
- Leveraging **big data from repositories** (GitHub, Jira, CI/CD logs) for more robust modeling.

Topic for discussion

Improving CI/CD in Software Development using Version Control Data and Bug Tracking Data

Context

- CI/CD (Continuous Integration and Continuous Deployment/Delivery) is now the **backbone of modern software engineering**
- Its effectiveness depends on:
  - How well we integrate code changes (from version control systems like Git).
  - How fast and reliably we detect and fix bugs (from bug tracking systems like Jira, Bugzilla).

- Combining **Version Control Data (VCS)** and **Bug Tracking Data (BTD)**, teams can build predictive and adaptive mechanisms to **improve the speed, reliability, and quality** of CI/CD pipelines.

Data Sources

Version Control Data (VCS)

- From Git, SVN, Mercurial, etc.:
- Commit frequency & size.
- Code churn (additions, deletions, modifications).
- Branching and merging patterns.
- Developer activity & ownership.
- Build/test outcomes linked to commits.

Data Sources (2)

Bug Tracking Data (BTD)

- From Jira, Bugzilla, Redmine, etc.:
- Bug density per module.
- Severity, priority, and type of defects.
- Resolution time (MTTR).
- Reopened bugs (recurrence issues).
- Historical link between features and defect introduction.

How these improve CI/CD?

Smarter Build Triggering

- Instead of running the *same* full test suite on every commit, pipelines can **prioritize tests** based on past defect-prone modules identified via VCS + BTD.
- Example: If a developer modifies a file with a history of frequent bug fixes, CI can run an **extended test suite**automatically.

Defect Prediction in Pipelines

- Using ML on commit metadata (e.g., churn, complexity) + historical bug data → flag "risky commits."
- CI/CD pipeline can enforce **extra code review or QA** for those commits before merging.

How these improve CI/CD? (2)

- **Release Risk Scoring**
- Combine unresolved bug counts, defect density trends, and commit instability to calculate a **release readiness score**.
- Prevents unstable builds from being promoted to production.

Continuous Feedback to Developers

- Developers get **real-time dashboards**:
  - Which modules are fragile.
  - Which commits are likely to break builds.
  - Which bug patterns are repeating.
- Encourages cleaner coding and disciplined commits.

How these improve CI/CD? (3)

Prioritizing Technical Debt Reduction

- Historical bug hotspots + high-churn modules → highlight candidates for **refactoring**.
- CI/CD pipelines can enforce “quality gates” (e.g., no new commits if code coverage drops in fragile modules).

Technical Solutions

- **Machine Learning Models**: Random Forest, Gradient Boosting, Neural Networks for defect prediction.
- **Process Mining**: Discover bottlenecks in bug resolution workflows.
- **NLP**: Analyze bug report text for clustering and categorization.
- **Anomaly Detection**: Spot unusual commit patterns likely to introduce instability.

Benefits

- **Higher Build Stability**
  - Fewer broken builds because risky commits are identified early.
- **Faster Feedback Loops**
  - Tests and checks focus on likely defect areas → pipelines run faster.
- **Improved Software Quality**
  - Historical defect-prone modules receive more scrutiny.
- **Cost Efficiency**
  - Less wasted compute on unnecessary tests.
  - Less wasted engineer time fixing avoidable regressions.
- **Cultural Benefits**
  - Developers become more aware of **how their coding behavior links to defects**, encouraging accountability.

Challenges

- **Data Integration**: VCS and bug tracking tools may not always be well-linked (e.g., commits not referencing bug IDs).
- **Data Quality**: Incomplete bug reports or vague commit messages reduce accuracy.
- **Pipeline Complexity**: Adding predictive steps must not slow down CI/CD.
- **Team Adoption**: Developers must trust and act on pipeline insights, not bypass them.

Real-World Examples

- **Microsoft**: Uses commit metadata + bug history to detect risky changes in Windows builds.
- **Google**: Applies ML on code reviews and bug data to decide auto-approval thresholds in CI pipelines.
- **Facebook**: Prioritizes tests dynamically based on commit impact analysis.

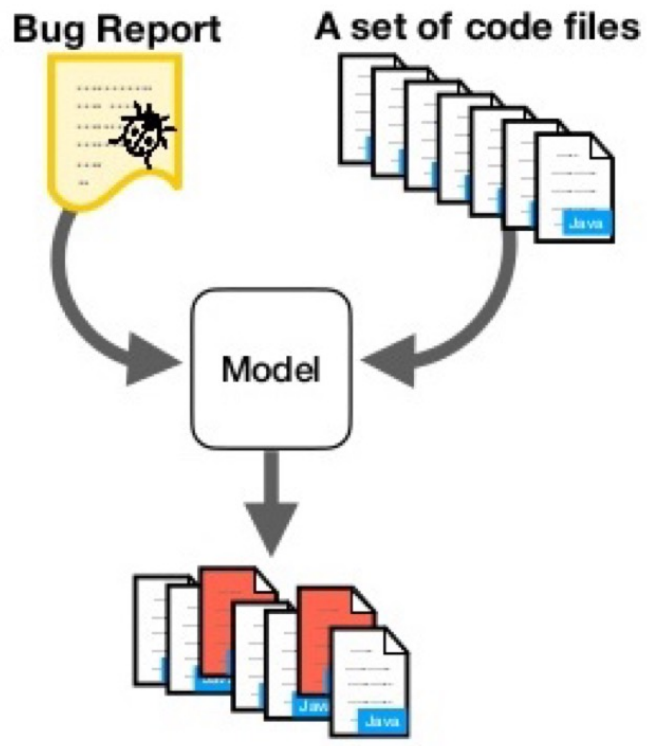
1st Project: Bug Localization

Bug tracking systems: Effectively manage bugs during the software development

Issue Type	Subject		Status	Start Date	Due date	Estimated Hours
Release	Release in-app announcement of new email design		Resolved	Jun. 26, 2017	Jun. 26, 2017	4
Release	Release blog for new email design		Open	Jun. 20, 2017	Jun. 26, 2017	20
Release	Release expense summary on mobile		Open	Jun. 20, 2017	Jun. 26, 2017	16
Release	Release Blog for English at June 27: New Design Mail		Resolved	Jun. 20, 2017	Jun. 26, 2017	16
Release	Release backlog-app 2017-06-13	(2/2)	Closed	Jun. 12, 2017	Jun. 13, 2017	6
Release	Release related dashboard URLs	(0/3)	In Progress	Jun. 12, 2017	Jun. 15, 2017	15
Release	Release backlog-app 2017-05-30		Closed	May. 29, 2017	May. 30, 2017	6
Release	Release view issue and project settings	(0/2)	Open	May. 10, 2017	May. 16, 2017	20
Release	Release backlog-app 2017-05-16	(1/1)	Closed	May. 10, 2017	May. 16, 2017	16
Release	Write a release blog for 2016-05-16		Closed	May. 10, 2017	May. 16, 2017	24

Automated Bug Localization Process

Bug Report 320437
Summary: PageBook toolbars lose the view menu drop down on a page change
Description:
If you start clean then the Outline view will have a view menu drop down but once you open an editor it doesn't...
Switching perspectives will bring it back (likely because we re-render the toolbar, causing the code in the RenderedToolbarRenderer to put it back...perhaps we can change the pagebook's update logic to do the same...



Bug reports vs Source files

Bug Report 320437
Summary: PageBook toolbars lose the view menu drop down on a page change
Description:
If you start clean then the Outline view will have a view menu drop down but once you open an editor it doesn't...
Switching perspectives will bring it back (likely because we re-render the toolbar, causing the code in the RenderedToolbarRenderer to put it back...perhaps we can change the pagebook's update logic to do the same...

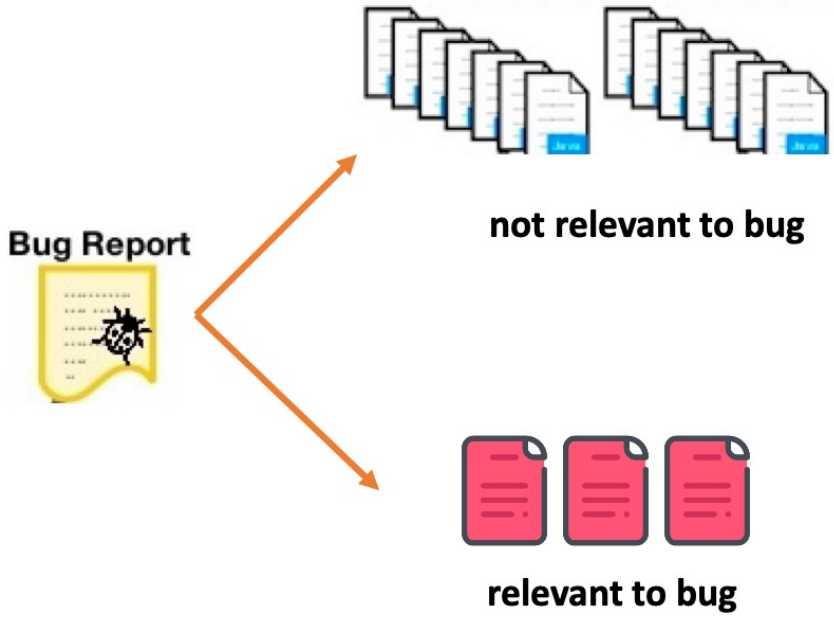
Natural Languages

Programming Languages

```
/**
 This class encapsulates the functionality necessary to manage
 stacks of parts in a 'lazy loading' manner. For these stacks only
 the currently 'active' child <b>most</b> be rendered so in this
 class we over ride that default behavior for processing the
 stack's contents to prevent all of the contents from being
 rendered, calling 'childAdded' instead. This not only saves time
 and SWT resources but is necessary in an IDE world where we must
 not arbitrarily cause plug-in loading.
 */
public abstract class LazyStackRenderer extends SWTPartRenderer {
  private EventHandler lazyLoader = new EventHandler() {
    public void handleEvent(Event event) {
      Object element =event.getProperty(UIEvents.EventTags.ELEMENT);
  if (!( element instanceof MGenericStack<?>))
      return;
  MGenericStack<MUIElement> stack = (MGenericStack <MUIElement>)
          element;
  LazyStackRenderer lsr= (LazyStackRenderer) stack.getRenderer();
        ...
}
```

Problem Statement

Bug Localization is formed as **Information Retrieval** or **Binary Classification**?



Which features can be extracted from data?

Meta-data from bug reports and source files

- Semantic similarity
- Lexical similarity
- Past buggy source files to a given bug report ➔ Similar Bug Reports
- Code change history
- Bug Fixing frequency

A very simple model

$\mathcal{B} = \{r_1, r_2, ..., r_N\}$: set of bug reports

$\mathcal{S} = \{s_1, s_2, ..., s_M\}$: a collection of source files

$V_d = \{w_1, w_2, ..., w_{\mathbb{N}}\}$: the vector representation of either a bug report or a source file

$$w_i = tf_i \times idf_i = (\log f_i + 1) \times (\log \frac{M}{n_i} + 1)$$

$$\mathcal{L}_{sim}(r, s) = \frac{\overrightarrow{V_r} \cdot \overrightarrow{V_s}}{\|\overrightarrow{V_r}\| \times \|\overrightarrow{V_s}\|}$$

More complicated models?

- Deep learning to learn non-linear relationship between a bug report and a source files
- Pre-trained code embedding models
- …

Evaluation metrics

Top-k accuracy measures the percentage of bug reports at which at least one buggy file is correctly found in the top k rank. Typically, the model is measured by top-1, top-5, top-10 and top-15 files.

Mean Reciprocal Rank (MRR) measures the average of the reciprocal rank for all bug reports. The reciprocal rank of a bug report is the inverse rank of its first correctly-located buggy file (Equation 11).

$$MRR = \frac{1}{N} \sum_{r \in \mathcal{B}} \frac{1}{first(r)} \tag{11}$$

Mean Average Precision (MAP) is used to evaluate the mean of all *average precision* values ($AvgPre(r)$) (Equations 12, 13 and 14).

$$MAP = \sum_{r \in \mathcal{B}} \frac{AvgPre(r)}{N} \tag{12}$$

where N is the number of bug reports in $\mathcal{B}$. The average precision of a bug report r is computed as follows:

$$AvgPre(r) = \sum_{k \in K} \frac{Pre_k(r)}{|K|} \tag{13}$$

where K is the list of top ranking source files in descending order of f values, with respect to r, $|K|$ denotes the number of source files in K. The precision at k-th rank, $Pre_k(r)$ is computed as the number of relevant source files over the top k files:

$$Pre_k(r) = \frac{\#\text{ of relevant source files in top } k}{k} \tag{14}$$

LLMs for Software Engineering

(end of lecture)

ONE LOVE. ONE FUTURE.